

# Il Manifesto del Secure Coding: I 10 Pilastri

## 1. Validare sempre gli Input (Zero Trust)

Mai fidarsi dei dati provenienti dall'esterno, siano essi form, parametri URL, cookie o file caricati.

- **Azione:** Implementa una validazione basata su "allow-list" (cosa è permesso) invece che "deny-list" (cosa è vietato). Verifica tipo, lunghezza, formato e intervallo.

## 2. Principio del Minimo Privilegio (PoLP)

Ogni processo, utente o modulo del codice deve operare utilizzando solo il set minimo di permessi necessari per completare il proprio lavoro.

- **Azione:** Se un'applicazione deve solo leggere da un database, non utilizzare un utente con permessi di scrittura o amministrazione.

## 3. Codifica e Sanificazione dell'Output

Per prevenire attacchi come il Cross-Site Scripting (XSS), i dati devono essere neutralizzati prima di essere visualizzati nel browser o inviati a un altro sistema.

- **Azione:** Usa librerie di escaping contestuale (HTML, JS, CSS) per assicurarti che il browser interpreti i dati come testo e non come codice eseguibile.

## 4. Gestione Sicura dell'Autenticazione e delle Sessioni

Le credenziali degli utenti e gli identificativi di sessione sono il bersaglio preferito dei cyber-criminali.

- **Azione:** Usa algoritmi di hashing forti (come Argon2 o bcrypt) per le password. Implementa il Multi-Factor Authentication (MFA) e assicuratevi che i cookie di sessione siano `HttpOnly` e `Secure`.

## 5. Difesa in Profondità (Layered Defense)

Non affidarti a un singolo meccanismo di sicurezza. Se uno strato fallisce, un altro deve essere pronto a intervenire.

- **Azione:** Combina firewall, validazione lato server, crittografia dei dati e monitoraggio costante.

## 6. Crittografia Ovunque (Dati in transito e a riposo)

I dati sensibili non devono mai viaggiare o essere memorizzati in chiaro.

- **Azione:** Usa protocolli moderni come TLS 1.3 per i dati in movimento e standard come AES-256 per i dati salvati su disco. Non inventare mai i tuoi algoritmi crittografici.

## 7. Gestione Sicura degli Errori e Logging

I messaggi d'errore non devono mai rivelare dettagli sull'infrastruttura (stack traces, versioni del database, percorsi dei file).

- **Azione:** Mostra messaggi generici all'utente e registra i dettagli tecnici in log protetti, utili per il debugging e l'analisi forense in caso di attacco.

## 8. Utilizzo di Componenti Terzi Sicuri

Le vulnerabilità spesso non sono nel tuo codice, ma nelle librerie e nei framework che utilizzi.

- **Azione:** Esegui regolarmente scansioni SCA (Software Composition Analysis) per identificare e aggiornare dipendenze con falle note (CVE).

## 9. Protezione dei Segreti (No Hardcoding)

Chiavi API, password di database e token non devono mai apparire nel codice sorgente o nei repository Git.

- **Azione:** Utilizza variabili d'ambiente o, meglio ancora, servizi di "Secret Management" (come Vault, AWS Secrets Manager o Azure Key Vault).

## 10. Sicurezza come Processo (Shift Left)

La sicurezza non è un check finale prima del rilascio, ma parte integrante del ciclo di vita del software (SDLC).

- **Azione:** Implementa Code Review focalizzate sulla sicurezza e test automatizzati (SAST/DAST) fin dalle prime fasi di sviluppo.

---

**Regola d'oro:** *Scrivi codice pensando che l'attaccante sia già dentro la tua rete. La semplicità è l'anima della sicurezza: un codice complesso è più difficile da auditare e nasconde più facilmente delle vulnerabilità.*

## Materiale di supporto:

### 1. OWASP Foundation (Progetti 2026)

OWASP rimane il punto di riferimento assoluto, ma nel 2026 ha esteso enormemente i suoi standard:

- [OWASP Top 10 for Agentic Applications](#): Essenziale se stai sviluppando software che integra agenti AI o LLM. Copre rischi come il *Tool Misuse* e la delega insicura dei permessi.
- [OWASP Smart Contract Top 10 \(2026\)](#): Se lavori in ambito Web3 o Blockchain.
- [OWASP ASVS](#): La lista di controllo tecnica definitiva per verificare che la tua applicazione sia sicura a livello "enterprise".

### 2. NIST - Secure Software Development Framework (SSDF)

Il NIST (SP 800-218) è diventato lo standard richiesto da molti governi e grandi aziende. È meno una guida di "codice" e più una guida di "**processo**" (come integrare la sicurezza in ogni commit).

- Sito: [NIST CSRC](#)
- 

## Apprendimento Pratico e Lab (Mani sul Codice)

### 3. AppSecEngineer & SecureFlag

Queste piattaforme sono le preferite nel 2026 per la formazione aziendale perché usano lab basati su browser con ambienti di sviluppo reali.

- [AppSecEngineer](#): Focalizzato molto su DevSecOps, Cloud Security e Kubernetes.
- [SecureFlag](#): Offre lab pratici dove devi letteralmente correggere il codice vulnerabile per superare l'esercizio.

### 4. PortSwigger Web Security Academy

È probabilmente la migliore risorsa **gratuita** al mondo per imparare l'attacco e la difesa web. I laboratori sono costantemente aggiornati con le nuove tecniche di exploit scoperte ogni anno.

- Sito: [portswigger.net/web-security](https://portswigger.net/web-security)
- 

## Strumenti di Analisi e Database Vulnerabilità

### 5. GitHub Security & CodeQL

GitHub è diventato un hub di sicurezza. Studiare come scrivere query **CodeQL** ti permette di automatizzare la ricerca di pattern insicuri nel tuo codice direttamente nelle GitHub Actions.

- **Sito:** [GitHub Security Advisories](#)

## 6. Snyk & Jit

Ideali per la **Supply Chain Security**. Ti aiutano a monitorare se le librerie (npm, pip, maven) che stai usando hanno vulnerabilità note (CVE) aggiornate in tempo reale.

- **Sito:** [Snyk Vulnerability Database](#)

---

### Consiglio per iniziare subito:

Consiglio l'[OWASP Secure Coding Dojo](#). È un progetto gratuito che ti guida attraverso le vulnerabilità più comuni (basate sulla lista SANS 25) con un approccio "attacco e difesa".